

Controle de Manutenção de Equipamentos Hospitalares Hospital Equipment Maintenance Control (HEMC)

Elias Martins de Almeida
Evelyn Victoria Araújo dos Santos
Guilherme Rodrigues Guedes
Gabriel Santoni Espindola

RESUMO

Sistema web de controle de manutenção de equipamentos hospitalares, com o objetivo de gerenciar a manutenção de máquinas e equipamentos, garantindo maior organização, segurança e confiabilidade dos processos. A aplicação permitirá o cadastro dos equipamentos, o registro de manutenções preventivas, corretivas e a geração de alertas automáticos.

Palavras-chave: Hospitalares, Manutenção, Gestão, Preventiva e Corretiva

ABSTRACT

A web-based system for managing the maintenance of hospital equipment, designed to manage the upkeep of machines and equipment, ensuring greater organization, safety, and reliability of processes. The application will allow for equipment registration, recording of preventive and corrective maintenance, and the generation of automatic alerts.

Keywords: Hospitals, Maintenance, Management, Preventive and Corrective

1 INTRODUÇÃO

A Engenharia Clínica é responsável pela gestão tecnológica dos equipamentos médico-hospitalares, garantindo que estejam disponíveis, seguros e em conformidade com as normas vigentes. Nesse contexto, os sistemas informatizados de gestão da manutenção, conhecidos como CMMS (*Computerized Maintenance Management System*), desempenham papel fundamental no controle e organização das atividades técnicas relacionadas aos equipamentos de saúde.

Esses sistemas permitem o cadastro detalhado dos dispositivos, o planejamento das manutenções preventivas, o gerenciamento das manutenções corretivas e o registro completo das intervenções realizadas. Além disso, possibilitam a geração de indicadores de desempenho que auxiliam na tomada de decisões estratégicas, contribuindo para a melhoria contínua dos processos e para a redução de falhas inesperadas.

A utilização estruturada desses sistemas fortalece a atuação da Engenharia Clínica, promovendo maior eficiência operacional, rastreabilidade das informações, suporte às auditorias e, principalmente, maior segurança para pacientes e profissionais de saúde.

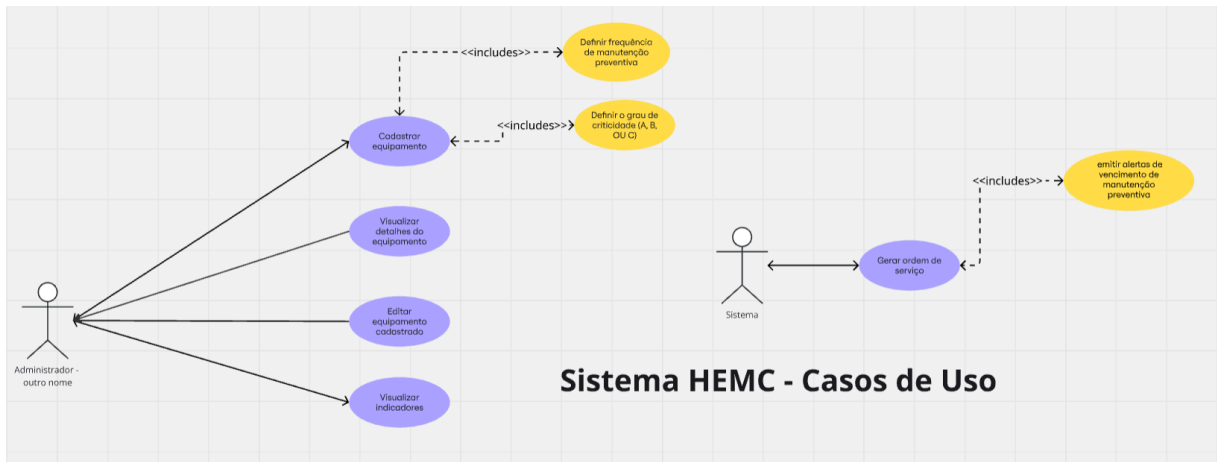
1.1 Objetivo

Gerenciar a manutenção de máquinas e equipamentos, garantindo maior organização, segurança e confiabilidade, focando principalmente na experiência do usuário que não é da área de T.I, uma problemática que vimos em um sistema que está presente no mercado atual, termos técnicos como “SQL” em teoria não precisam ser conhecidos pelos técnicos de engenharia clínica, porém estavam presentes no sistema analisado, ou seja nosso objetivo eliminar essa problemática e trazer um sistema eficiente que permite o cadastro detalhado dos dispositivos, o planejamento das manutenções preventivas, o gerenciamento das manutenções corretivas e o registro completo das intervenções realizadas.

2 METODOLOGIA

2.1 O projeto pode ser visto no nosso repositório do github:
https://github.com/Lyndacrebs/Projeto_Hemec.git

2.2 Visão Geral



2.2.1 Ator 1 - Administrador do Sistema: O Administrador representa o usuário responsável pela gestão dos equipamentos hospitalares dentro do sistema. Esse ator possui permissões administrativas, sendo encarregado do cadastro, atualização e acompanhamento das informações relacionadas aos equipamentos e aos indicadores de manutenção.

O administrador interage diretamente com as funcionalidades principais de gerenciamento, garantindo que os dados necessários para o funcionamento do sistema estejam corretamente registrados.

2.2.1.1 Caso de Uso - Cadastrar Equipamento:

O caso de uso **Cadastrar Equipamento** permite ao administrador inserir novos equipamentos hospitalares no sistema. Durante esse processo, são registradas informações essenciais para o controle de manutenção, possibilitando o acompanhamento completo do ciclo de vida do equipamento.

Esse cadastro é fundamental para que o sistema possa posteriormente realizar o controle das manutenções preventivas e corretivas.

2.2.1.1.1 Casos incluídos (<<include>>):

Este caso de uso inclui obrigatoriamente as seguintes funcionalidades:

- **Definir frequência de manutenção preventiva**

Permite estabelecer a periodicidade das manutenções preventivas do equipamento, podendo ser configurada conforme a necessidade institucional (por exemplo: mensal ou trimestral). Essa informação é utilizada pelo sistema para o controle automático dos prazos de manutenção.

- **Definir grau de criticidade (A, B ou C)**

Permite classificar o equipamento de acordo com seu nível de criticidade assistencial:

- **A** — Alta criticidade (impacto direto no atendimento ao paciente);
- **B** — Criticidade moderada;
- **C** — Baixa criticidade.

Essa classificação auxilia na priorização das manutenções e na gestão dos riscos operacionais.

2.2.1.2 **Caso de Uso** - Visualizar Detalhes do Equipamento:

Este caso de uso possibilita ao administrador consultar todas as informações cadastradas de um equipamento específico, incluindo dados técnicos, localização, criticidade e histórico relacionado às manutenções realizadas. A funcionalidade contribui para a análise rápida da situação operacional dos equipamentos.

2.2.1.3 **Caso de Uso** - Editar Equipamento Cadastrado:

Permite ao administrador atualizar informações previamente registradas no sistema, garantindo que alterações como mudança de localização, ajustes na criticidade ou atualização de dados técnicos possam ser realizadas sempre que necessário. Essa funcionalidade assegura a integridade e a atualização contínua dos dados.

2.2.1.4 **Caso de Uso** - Visualizar Indicadores:

O caso de uso **Visualizar Indicadores** possibilita ao administrador acompanhar métricas e indicadores de desempenho relacionados às manutenções dos equipamentos hospitalares.

Entre os indicadores apresentados podem estar:

- quantidade de manutenções realizadas;
- equipamentos com manutenção vencida;
- status geral dos equipamentos;
- desempenho das atividades de manutenção.

Essas informações auxiliam na tomada de decisão e no planejamento das ações preventivas.

2.2.2 Ator 2 - Sistema: O ator **Sistema** representa processos automáticos executados sem interação humana direta. Ele é responsável por ações programadas baseadas em regras previamente definidas, como verificação de prazos e geração automática de atividades.

2.2.2.1 **Caso de Uso** - Gerar Ordem de Serviço:

Este caso de uso descreve o processo automático em que o sistema cria uma **Ordem de Serviço (OS)** quando identifica a necessidade de manutenção preventiva de um equipamento, considerando a frequência previamente definida no cadastro.

A ordem de serviço representa uma tarefa formal que deverá ser executada pela equipe responsável pela manutenção.

2.2.2.1.1 Caso incluído (<<include>>):

Sempre que uma ordem de serviço é gerada, o sistema emite automaticamente um alerta informando aos usuários responsáveis que uma manutenção preventiva está próxima do vencimento ou já se encontra vencida. Esse mecanismo garante maior controle e reduz o risco de falhas operacionais decorrentes da falta de manutenção.

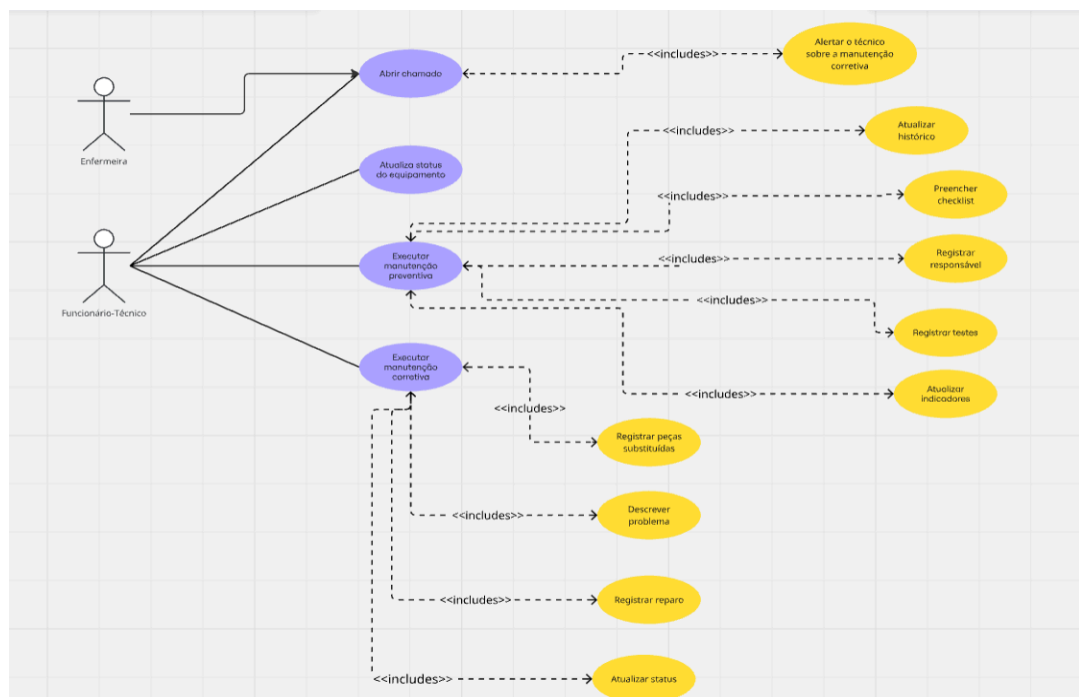


Figura 1: Diagrama de Caso de Uso

2.2.3 Ator 3 - Enfermeira: A **Enfermeira** representa o profissional assistencial que utiliza os equipamentos hospitalares no cotidiano e pode identificar falhas ou irregularidades durante sua utilização.

Esse ator possui acesso à funcionalidade de abertura de chamados, permitindo comunicar à equipe técnica a necessidade de manutenção corretiva.

2.2.3.1 Caso de Uso - Abrir Chamado:

O caso de uso **Abrir Chamado** permite que a enfermeira registre uma solicitação de manutenção quando identificar problemas em um equipamento. Durante esse processo, o sistema possibilita o envio de informações relevantes para análise técnica.

2.2.3.1.1 **Caso incluído** (<<include>>):

- **Alertar o técnico sobre a manutenção corretiva**

Após a abertura do chamado, o sistema envia automaticamente uma notificação ao técnico responsável, garantindo agilidade no atendimento da ocorrência.

2.2.4 Ator 4 - Funcionário Técnico: O **Funcionário Técnico** representa o profissional responsável pela execução das atividades de manutenção dos equipamentos hospitalares, tanto preventivas quanto corretivas. Esse ator interage diretamente com funcionalidades relacionadas à execução dos serviços e ao registro das intervenções realizadas.

2.2.4.1 **Caso de Uso** - Abrir Chamado:

O caso de uso **Abrir Chamado** está associado tanto à enfermeira quanto ao funcionário técnico, pois ambos os atores possuem permissão para registrar solicitações de manutenção no sistema. Essa associação representa que diferentes perfis de usuários podem iniciar o processo de manutenção, garantindo maior agilidade na comunicação de falhas e necessidades de intervenção nos equipamentos hospitalares.

2.2.4.2 **Caso de Uso** - Executar Manutenção Preventiva:

Este caso de uso descreve o processo de manutenção planejada, realizada periodicamente conforme a frequência definida no cadastro do equipamento.

A manutenção preventiva tem como objetivo evitar falhas e garantir o funcionamento seguro dos equipamentos.

2.2.4.2.1 **Casos incluídos** (<<include>>):

- **Preencher checklist:**
Registro dos testes de segurança e calibração realizados durante a manutenção.
- **Registrar responsável:**
Identificação do profissional que executou a manutenção, assegurando rastreabilidade.
- **Registrar testes:**
Inserção dos resultados obtidos nos testes técnicos realizados.
- **Atualizar histórico:**
Armazenamento automático da manutenção no histórico do equipamento.
- **Atualizar indicadores:**
Atualização dos indicadores de desempenho exibidos na tela inicial do sistema.

2.2.4.3 **Caso de Uso** - Executar Manutenção Corretiva:

O caso de uso **Executar Manutenção Corretiva** representa a intervenção realizada quando um equipamento apresenta falha ou defeito.

Esse processo ocorre após a abertura de um chamado e envolve o diagnóstico e reparo do equipamento.

2.2.4.3.1 **Casos incluídos** (<<include>>):

- **Descrever problema:**
Registro detalhado da falha identificada.
- **Registrar reparo:**
Documentação das ações realizadas para correção do problema.
- **Registrar peças substituídas:**
Identificação dos componentes trocados durante o reparo.
- **Atualizar status:**
Alteração do estado operacional do equipamento após a manutenção.

2.2.4.4 **Caso de Uso** - Atualizar Status do Equipamento:

Permite ao técnico modificar o status do equipamento conforme sua condição atual, podendo indicar, por exemplo:

- Em funcionamento;
- Em manutenção;
- Aguardando reparo;
- Fora de operação.

Essa funcionalidade contribui para o acompanhamento em tempo real da disponibilidade dos equipamentos.

2.3 **Lista de Requisitos**

2.3.1 **Cadastro e configuração:**

REQ1 – O sistema permitirá o cadastro de equipamentos preenchendo os seguintes campos: identificação patrimonial (valor do equipamento), fabricante, modelo, número de série, localização, tipo de equipamento, grau de criticidade, data de aquisição e frequência de manutenção.

REQ2 – O sistema permitirá definir três graus de criticidade para os equipamentos:

- A (alta criticidade),

- B (criticidade moderada),
- C (baixa criticidade).

REQ3 – O sistema permitirá definir a frequência de manutenção preventiva do equipamento como:

- mensal ou trimestral.

REQ4 – O sistema permitirá visualizar a lista de equipamentos cadastrados.

REQ5 – O sistema permitirá editar as informações de equipamentos já cadastrados.

REQ6 – O sistema permitirá excluir equipamentos cadastrados.

REQ7 – O sistema permitirá visualizar os detalhes completos de um equipamento selecionado.

2.3.2 Ordem de Serviço e Alertas:

REQ8 – O sistema deverá gerar automaticamente ordens de serviço de manutenção preventiva conforme a frequência definida para cada equipamento.

REQ9 – O sistema deverá emitir alertas de vencimento de manutenção preventiva quando a data programada estiver próxima ou vencida.

REQ10 – O sistema deverá exibir os alertas na tela inicial (home) do sistema.

2.3.3 Manutenção Preventiva:

REQ11 – O sistema permitirá registrar a execução de manutenção preventiva vinculada a um equipamento.

REQ12 – O sistema deverá disponibilizar um checklist contendo testes de segurança e calibração.

REQ13 – O sistema permitirá informar se houve substituição de componentes durante a manutenção preventiva.

REQ14 – O sistema deverá registrar automaticamente a manutenção preventiva no histórico do equipamento.

REQ15 – O sistema permitirá registrar o nome e o setor do responsável pela manutenção.

REQ16 – O sistema deverá atualizar automaticamente os indicadores de desempenho exibidos na tela inicial após o registro da manutenção.

REQ17 – O sistema poderá gerar um alerta público informando a realização da manutenção preventiva.

2.3.4 Manutenção Corretiva:

REQ18 – O sistema permitirá abrir um chamado de manutenção corretiva a partir da tela inicial ou da visualização do equipamento.

REQ19 – O sistema permitirá registrar a descrição do problema, setor, unidade e nível de urgência (criticidade).

REQ20 – O sistema permitirá registrar os serviços realizados durante o reparo.

REQ21 – O sistema permitirá registrar peças substituídas durante a manutenção corretiva.

REQ22 – O sistema permitirá registrar os testes finais realizados após o reparo.

REQ23 – O sistema deverá anexar as informações do reparo ao histórico do equipamento.

REQ24 – O sistema permitirá atualizar o status do equipamento (ex.: em manutenção, funcionando, aguardando peça, finalizado).

REQ25 – O sistema deverá registrar a manutenção corretiva com identificação diferente da manutenção preventiva no histórico.

2.4 Software Desenvolvimento

Para dar início à fase de prototipagem, conforme proposto na abordagem de Design Thinking apresentada pelo professor Emerson Cruz, o grupo optou pelo desenvolvimento de um protótipo inicial em HTML e CSS com o auxílio de Inteligência Artificial. Essa decisão foi tomada com base nos requisitos previamente definidos nas etapas anteriores do projeto, buscando transformar ideias abstratas em uma representação visual inicial do sistema.

Durante a construção do prompt utilizado na IA, a equipe se preocupou em ser o mais específica possível quanto aos resultados esperados, detalhando estrutura, funcionalidades e estilo visual desejado. Como resultado, foi gerado um arquivo HTML que serviu como ponto de partida para análises internas. A partir desse material, o grupo realizou discussões críticas para avaliar quais elementos deveriam ser mantidos, ajustados ou descartados, promovendo assim um refinamento colaborativo do protótipo inicial.

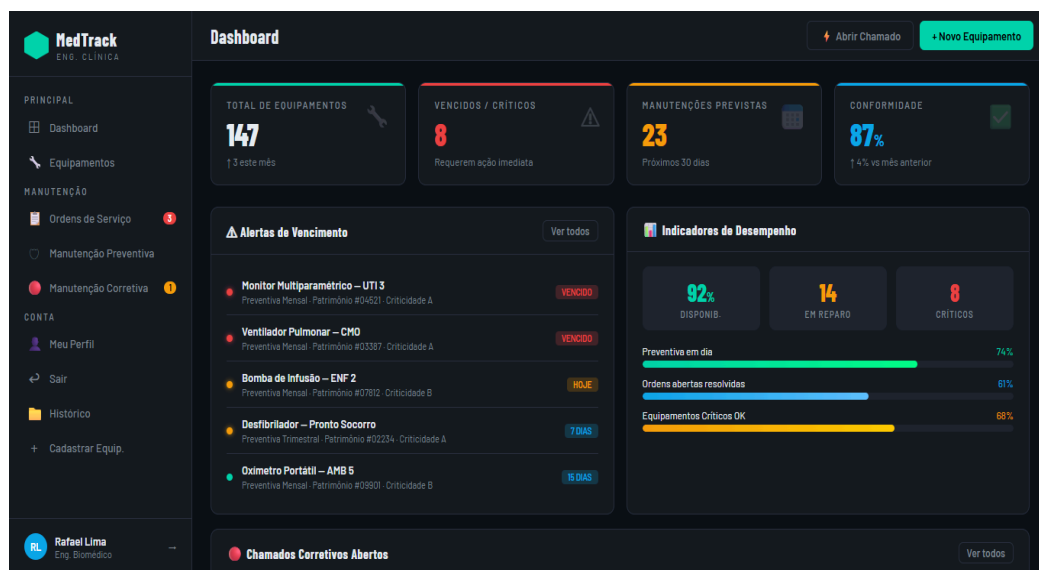


Figura 3: Foto do protótipo gerado por IA

Na sequência, tornou-se necessário definir uma ferramenta mais robusta e adequada para a evolução da prototipagem e visualização do produto final. Nesse contexto, foi escolhida a plataforma Figma, uma ferramenta colaborativa baseada em nuvem amplamente utilizada para criação de interfaces e protótipos de aplicações web e mobile. A escolha do Figma permitiu ao grupo trabalhar de forma simultânea e organizada, facilitando a comunicação entre os membros e garantindo maior fidelidade na representação do sistema.

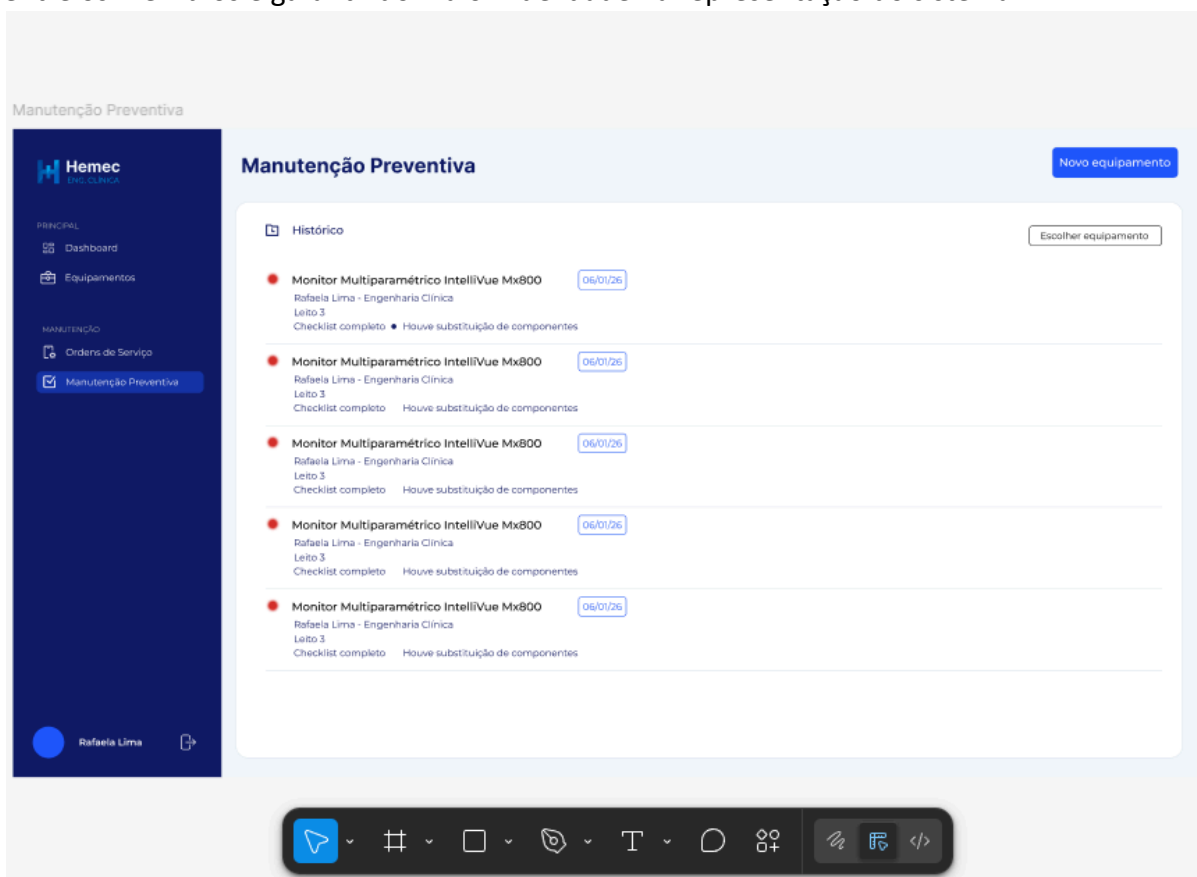


Figura 4: Foto do protótipo feito no Figma

Durante a etapa de modelagem no Figma, todos os aspectos relacionados ao design da interface foram cuidadosamente discutidos. Isso incluiu a definição da identidade visual do sistema, como a escolha das tipografias (utilizando a fonte Montserrat para os textos e Inter para os títulos), bem como a seleção da paleta de cores, com predominância do azul, visando transmitir confiança e clareza ao usuário. Além disso, o desenvolvimento da logomarca passou por diversas iterações, até que se chegasse a uma versão final capaz de representar adequadamente os objetivos e a proposta do sistema.



Figura 5: Protótipo da logo



Figura 6: Logo final

Outro fator relevante para a escolha do Figma foi sua utilidade como referência para a etapa de desenvolvimento. O protótipo elaborado na ferramenta passou a servir como um modelo visual a ser seguido na implementação em HTML e CSS, contribuindo também de forma indireta para o desenvolvimento do backend. Dessa forma, o Figma não apenas auxiliou na organização estética do sistema, mas também atuou como um guia funcional, permitindo que a equipe tivesse uma visão clara e estruturada de todos os elementos e funcionalidades que deveriam ser implementados.

Para dar continuidade ao desenvolvimento do software, foi realizada a definição das tecnologias que seriam utilizadas ao longo do projeto. Essa etapa incluiu a escolha das linguagens de programação, ferramentas de desenvolvimento, banco de dados e a API

responsável pela comunicação entre o frontend e o backend, garantindo uma arquitetura coerente e funcional.

A seguir, são apresentadas as principais tecnologias adotadas pelo grupo e suas respectivas finalidades no sistema:

- **HTML (HyperText Markup Language):** utilizado para a estruturação das páginas web, sendo responsável pela organização dos elementos que compõem a interface do sistema.
- **CSS (Cascading Style Sheets):** empregado na estilização das páginas, permitindo a definição de cores, tipografia, espaçamentos e layout, contribuindo diretamente para a experiência visual do usuário.
- **JavaScript:** utilizado para adicionar interatividade às páginas, possibilitando a manipulação de elementos HTML e CSS, além de viabilizar a comunicação com o backend por meio de requisições.
- **Python:** linguagem escolhida para o desenvolvimento do backend, sendo responsável pela implementação das regras de negócio e processamento das informações do sistema.
- **Fast API:** framework utilizado para a construção da API do sistema, permitindo a criação de rotas e a comunicação eficiente entre o frontend e o backend, com alto desempenho.
- **SQLite:** banco de dados relacional adotado para o armazenamento das informações do sistema, escolhido por sua leveza, simplicidade de configuração e integração com aplicações em Python.
- **GitHub:** Versionamento do sistema e serviço de armazenamento na nuvem
- **Trello:** Ferramenta de gestão de projetos e tarefas online.

Após a definição das tecnologias a serem utilizadas, o grupo realizou a distribuição das responsabilidades entre os integrantes, com foco principal na separação entre as áreas de frontend e backend. Essa divisão teve como objetivo otimizar o desenvolvimento, permitindo que cada membro se concentrasse em tarefas específicas, além de facilitar a organização e o progresso do projeto.

A seguir, estão descritas as atribuições de cada integrante da equipe:

- **Evelyn Victoria:** responsável pelo desenvolvimento do frontend, incluindo a modelagem das interfaces na plataforma Figma, bem como o gerenciamento de versionamento utilizando o Git e o GitHub.
- **Elias Martins:** responsável pela implementação do backend, com foco na construção da API utilizando o framework Fast API.
- **Gabriel Santoni:** responsável pela modelagem e gerenciamento do banco de dados, além de atuar no desenvolvimento do backend e na elaboração da documentação do projeto.
- **Guilherme Guedes:** responsável pelo desenvolvimento do frontend e pela modelagem das interfaces no Figma.

2.3.1 Desenvolvimento do Front-end

Para o desenvolvimento do Front-end do projeto Hemec, foi utilizada a linguagem HTML para estruturar as páginas do sistema e organizar os elementos presentes na interface. Durante a criação das páginas, foram empregados recursos como divs, inputs e classes, permitindo uma melhor separação e organização dos componentes do sistema.

Essas estruturas possibilitaram a criação de uma interface mais organizada e de fácil compreensão para o usuário, facilitando a navegação e a disposição das informações dentro da plataforma. Os inputs foram aplicados nas áreas de preenchimento e interação do usuário com o sistema, enquanto as divs auxiliaram na organização do código. Já as classes contribuíram para a personalização e padronização dos elementos visuais no CSS.

Para a estilização da interface, foi utilizada a linguagem CSS, responsável pela definição do design e da aparência visual do sistema. Foram aplicadas propriedades de alinhamento, espaçamento, dimensionamento e organização dos elementos, contribuindo para uma interface mais moderna e funcional.

Também foi utilizado o recurso Flexbox para auxiliar no posicionamento e alinhamento dos elementos da página, permitindo uma melhor distribuição do conteúdo.

Além disso, foram definidas cores específicas para manter um padrão visual harmonioso. As principais cores utilizadas foram branco, #131b62 e #3754ed.

Também foram escolhidas as fontes Inder para os títulos e Montserrat para os textos, buscando melhorar a legibilidade, a organização e a estética da interface.

2.5 Ferramenta de Organização

O quadro Trello foi utilizado para organizar e acompanhar o desenvolvimento do Projeto Integrador. As tarefas foram divididas em colunas de acordo com as áreas do projeto, como acesso rápido, front-end, back-end, documentação e definição de escopo.



Figura 7: Quadro do Trello

Na coluna de acesso rápido, foram adicionados links importantes para ferramentas utilizadas pela equipe, como GitHub, Figma e documentos do projeto, facilitando o acesso rápido aos recursos necessários.

As colunas de Front-End e Back-End concentram as atividades relacionadas ao desenvolvimento do sistema, incluindo criação de interfaces em HTML/CSS, prototipação no Figma, modelagem do banco de dados e programação em Python. Cada cartão representa uma tarefa específica, contendo responsáveis, prazo de entrega e progresso da atividade.

A coluna de documentação foi utilizada para armazenar materiais importantes do projeto, como o DER (Diagrama Entidade-Relacionamento) do banco de dados. Já a coluna “Definir o escopo” reúne etapas relacionadas ao planejamento e estruturação do trabalho acadêmico, como levantamento de requisitos, diagramas, resultados, conclusão e referências.

O uso do Trello permitiu uma melhor organização da equipe, acompanhamento das tarefas e divisão de responsabilidades, contribuindo para o controle do andamento do projeto.

Além de auxiliar na organização da equipe, o quadro se relaciona com conceitos básicos da Engenharia de Software, como planejamento, divisão de tarefas, controle de atividades e rastreabilidade do projeto.

O uso do Trello também se aproxima da metodologia Scrum, funcionando como um quadro de gerenciamento de Sprint, onde as tarefas possuem responsáveis, prazos e acompanhamento de progresso. Isso contribuiu para melhorar a comunicação, colaboração e controle do desenvolvimento do sistema.

2.6 Banco de dados

Objetivo do Banco

O objetivo da modelagem de dados é implementar um banco de dados com foco na eficiência, segurança e integridade das informações, normalizado até a Terceira Forma Normal (3FN), de modo a eliminar redundâncias e garantir a confiabilidade do histórico de manutenções. O banco foi projetado para o SGBD SQLite, escolhido por sua leveza e integração com a aplicação em Python/FastAPI, permitindo o controle automático dos prazos de manutenção e a geração de indicadores de desempenho.

Regras de negócio

- O grau de criticidade de um equipamento deve ser obrigatoriamente A (alta), B (moderada) ou C (baixa).
- A frequência de manutenção preventiva deve ser mensal ou trimestral.
- Todo equipamento deve estar vinculado a um único setor, fabricante e tipo.
- Um setor pertence a uma única unidade hospitalar.

- Um chamado pode ser aberto tanto por uma enfermeira quanto por um funcionário técnico.
- Uma ordem de serviço preventiva é gerada automaticamente, sem chamado de origem.
- Uma ordem de serviço corretiva é sempre originada a partir de um chamado.
- Toda manutenção é classificada como preventiva ou corretiva (especialização total e disjunta).
- Toda manutenção possui um único responsável (funcionário técnico) e está vinculada a um único equipamento.
- O histórico de um equipamento é composto pelo conjunto de suas manutenções, diferenciando-se preventivas de corretivas.
- Não é permitido excluir o histórico de manutenções de um equipamento.
- O checklist de testes (segurança e calibração) é exclusivo das manutenções preventivas.
- A quantidade de uma peça substituída deve ser sempre maior que zero.
- Os indicadores de desempenho são dados derivados, calculados a partir das manutenções, e não são armazenados.

Definição das entidades e seus atributos

Nos itens seguintes, serão apresentadas as definições das tabelas, incluindo tipos de dados, comprimento, restrições (constraints), valor padrão e o propósito de cada coluna. Os tipos correspondem aos tipos nativos do SQLite (INTEGER, TEXT, REAL).

Tabela unidade

A tabela unidade armazena as unidades hospitalares. Cada unidade agrupa um ou mais setores.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
unidade	id_unidade	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador da unidade, gerado automaticamente
	nome_unidade	TEXT	Variável	NOT NULL	N/D	Nome da unidade hospitalar

Tabela setor

A tabela setor representa os setores de uma unidade hospitalar. Cada setor pertence a uma única unidade, referenciada pela chave estrangeira id_unidade.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
setor	id_setor	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do setor, gerado automaticamente
	nome_setor	TEXT	Variável	NOT NULL	N/D	Nome do setor
	id_unidade	INTEGER	8 bytes	FK, NOT NULL	N/D	Unidade à qual o setor pertence

Tabela fabricante

A tabela fabricante armazena os fabricantes dos equipamentos. Foi extraída da tabela equipamento para evitar repetição de dados e atender à 3FN.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
fabricante	id_fabricante	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do fabricante
	nome_fabricante	TEXT	Variável	NOT NULL	N/D	Nome do fabricante

Tabela tipo_equipamento

A tabela tipo_equipamento é um catálogo dos tipos de equipamento (por exemplo: monitor multiparamétrico, ventilador pulmonar).

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
tipo_equipamento	id_tipo	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do tipo de equipamento
	nome_tipo	TEXT	Variável	NOT NULL	N/D	Nome do tipo de equipamento
	descricao	TEXT	Variável	—	N/D	Descrição complementar do tipo

Tabela usuario

A tabela usuario unifica os atores Administrador, Enfermeira e Funcionário Técnico, diferenciando-os pelo atributo perfil. Cada usuário pertence a um setor.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
usuario	id_usuario	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do usuário

	nome	TEXT	Variável	NOT NULL	N/D	Nome do usuário
	perfil	TEXT	Variável	NOT NULL, CHECK	N/D	Perfil: admin, enfermeira ou tecnico
	id_setor	INTEGER	8 bytes	FK, NOT NULL	N/D	Setor ao qual o usuário pertence

Tabela equipamento

A tabela equipamento é o núcleo do sistema. Armazena os dados patrimoniais e técnicos do equipamento, seu grau de criticidade, a frequência de manutenção preventiva e o status operacional atual.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
equipamento	id_equipamento	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do equipamento
	num_patrimonio	TEXT	Variável	NOT NULL, UNIQUE	N/D	Número de patrimônio do equipamento
	modelo	TEXT	Variável	—	N/D	Modelo do equipamento
	num_serie	TEXT	Variável	—	N/D	Número de série
	data_aquisicao	TEXT	10 chars	—	N/D	Data de aquisição (AAAA-MM-DD)
	valor_aquisicao	REAL	8 bytes	—	N/D	Valor de aquisição
	grau_criticidade	TEXT	1 char	NOT NULL, CHECK	N/D	Criticidade: A, B ou C
	frequencia_preventiva	TEXT	Variável	NOT NULL, CHECK	N/D	Periodicidade: mensal ou trimestral
	status_atual	TEXT	Variável	NOT NULL, CHECK	em_funcionamento	Estado operacional atual
	id_setor	INTEGER	8 bytes	FK, NOT NULL	N/D	Setor onde o equipamento está alocado

	id_tipo	INTEGER	8 bytes	FK, NOT NULL	N/D	Tipo do equipamento
	id_fabricante	INTEGER	8 bytes	FK, NOT NULL	N/D	Fabricante do equipamento

Tabela chamado

A tabela chamado registra as solicitações de manutenção corretiva abertas por enfermeiras ou técnicos sobre um equipamento.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
chamado	id_chamado	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do chamado
	descricao_problema	TEXT	Variável	NOT NULL	N/D	Descrição do problema relatado
	nivel_urgencia	TEXT	1 char	NOT NULL, CHECK	N/D	Nível de urgência: A, B ou C
	data_abertura	TEXT	19 chars	NOT NULL	datetime('now')	Data e hora de abertura
	status_chamado	TEXT	Variável	NOT NULL, CHECK	aberto	Situação: aberto, em_andamento, fechado
	id_equipamento	INTEGER	8 bytes	FK, NOT NULL	N/D	Equipamento relacionado ao chamado
	id_usuario_abertura	INTEGER	8 bytes	FK, NOT NULL	N/D	Usuário que abriu o chamado
	id_setor	INTEGER	8 bytes	FK, NOT NULL	N/D	Setor de origem do chamado

Tabela ordem_servico

A tabela ordem_servico representa as ordens de serviço. As preventivas são geradas automaticamente (id_chamado nulo); as corretivas têm origem em um chamado.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
ordem_servico	id_os	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador da ordem de serviço

	tipo	TEXT	Variável	NOT NULL, CHECK	N/D	Tipo: preventiva ou corretiva
	data_geracao	TEXT	19 chars	NOT NULL	datetime('now')	Data de geração da OS
	data_programada	TEXT	10 chars	—	N/D	Data programada (base dos alertas)
	status_os	TEXT	Variável	NOT NULL, CHECK	aberta	Situação: aberta, em_execucao, concluida, cancelada
	id_equipamento	INTEGER	8 bytes	FK, NOT NULL	N/D	Equipamento da ordem de serviço
	id_chamado	INTEGER	8 bytes	FK	NULL	Chamado de origem (nulo se preventiva)

Tabela manutencao

A tabela manutencao é o supertipo da especialização. Armazena os dados comuns a toda manutenção e constitui o histórico do equipamento. O campo tipo é o discriminador entre preventiva e corretiva.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
manutencao	id_manutencao	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador da manutenção
	tipo	TEXT	Variável	NOT NULL, CHECK	N/D	Discriminador : preventiva ou corretiva
	data_execucao	TEXT	19 chars	NOT NULL	datetime('now')	Data de execução da manutenção
	id_equipamento	INTEGER	8 bytes	FK, NOT NULL	N/D	Equipamento que sofreu a manutenção
	id_responsavel	INTEGER	8 bytes	FK, NOT NULL	N/D	Usuário técnico responsável
	id_os	INTEGER	8 bytes	FK, NOT NULL, UNIQUE	N/D	Ordem de serviço que originou a manutenção

Tabela manutencao_preventiva

Subtipo da manutenção. A chave primária é a mesma do supertipo (id_manutencao atua como PK e FK simultaneamente).

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
manutencao_preventiva	id_manutencao	INTEGER	8 bytes	PK, FK	N/D	Identificador herdado de manutenção
	houve_substituicao	INTEGER	1 byte	NOT NULL, CHECK (0/1)	0	Indica se houve substituição de peças
	checklist_concluido	INTEGER	1 byte	NOT NULL, CHECK (0/1)	0	Indica se o checklist foi concluído

Tabela manutencao_corretiva

Subtipo da manutenção referente às intervenções corretivas. Herda a chave primária do supertipo.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
manutencao_corretiva	id_manutencao	INTEGER	8 bytes	PK, FK	N/D	Identificador herdado de manutenção
	descricao_reparo	TEXT	Variável	—	N/D	Serviços realizados no reparo
	testes_finais	TEXT	Variável	—	N/D	Testes realizados após o reparo
	status_resultante	TEXT	Variável	CHECK	N/D	Status do equipamento após a correção

Tabela peca

Catálogo das peças e componentes que podem ser substituídos nas manutenções.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
peca	id_peca	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador da peça

	nome_peca	TEXT	Variável	NOT NULL	N/D	Nome da peça
	codigo	TEXT	Variável	—	N/D	Código de referência da peça

Tabela peca_substituida

Tabela associativa que resolve o relacionamento N:M entre manutencao e peca. O atributo quantidade é próprio do relacionamento.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
peca_substituida	id_manutencao	INTEGER	8 bytes	PK, FK	N/D	Manutenção em que a peça foi trocada
	id_pecas	INTEGER	8 bytes	PK, FK	N/D	Peça substituída
	quantidade	INTEGER	8 bytes	NOT NULL, CHECK (>0)	1	Quantidade de unidades substituídas

Tabela checklist_item

Catálogo dos itens de checklist (testes de segurança e calibração) executados nas manutenções preventivas.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
checklist_item	id_item	INTEGER	8 bytes	PK, NOT NULL	AUTOINCREMENT	Identificador do item de checklist
	descricao_teste	TEXT	Variável	NOT NULL	N/D	Descrição do teste
	tipo_teste	TEXT	Variável	NOT NULL, CHECK	N/D	Tipo: segurança ou calibração

Tabela manutencao_checklist

Tabela associativa que resolve o relacionamento N:M entre manutencao_preventiva e checklist_item, registrando o resultado de cada teste.

Tabela	Nome da Coluna	Tipo de Dado	Comprimento	Restrições	Valor Padrão	Descrição
manutencao_checklist	id_manutencao	INTEGER	8 bytes	PK, FK	N/D	Manutenção preventiva associada

	id_item	INTEGER	8 bytes	PK, FK	N/D	Item de checklist verificado
	resultado	TEXT	Variável	NOT NULL, CHECK	N/D	Resultado: aprovado, reprovado, nao_aplicavel
	observacao	TEXT	Variável	—	N/D	Observações sobre o teste

Relacionamentos entre as entidades

Este capítulo detalha as relações entre as entidades que compõem o modelo. A tabela a seguir descreve cada um desses relacionamentos e suas respectivas finalidades.

Tabela	Relacionamento	Nome do Relacionamento	Descrição
unidade	setor	Tem	Uma unidade possui vários setores
setor	usuario	Pertence	Um usuário pertence a um setor
setor	equipamento	Aloca	Um equipamento é alocado em um setor
equipamento	tipo_equipamento	Classifica-se	Cada equipamento possui um tipo
equipamento	fabricante	Fabricado por	Cada equipamento possui um fabricante
equipamento	manutencao	Sofre	Um equipamento sofre várias manutenções
usuario	chamado	Abre	Um usuário abre vários chamados
usuario	manutencao	Executa	Um técnico executa várias manutenções
chamado	ordem_servico	Origina	Um chamado origina uma OS corretiva
ordem_servico	manutencao	Gera	Uma OS gera uma manutenção
manutencao	manutencao_preventiva / manutencao_corretiva	Especializa	Especialização total e disjunta (t,d)
manutencao	peca	Substitui	Relacionamento N:M de peças substituídas
manutencao_preventiva	checklist_item	Verifica	Relacionamento N:M do checklist

Modelo Conceitual

Nesta seção, apresentamos o Modelo Conceitual do banco de dados do sistema HEMC. Este modelo representa as entidades principais (como Equipamento, Manutenção, Chamado e Ordem de Serviço) e as cardinalidades que regem suas interações, incluindo a especialização da entidade Manutenção em preventiva e corretiva. A figura a seguir ilustra visualmente como essas informações se conectam.

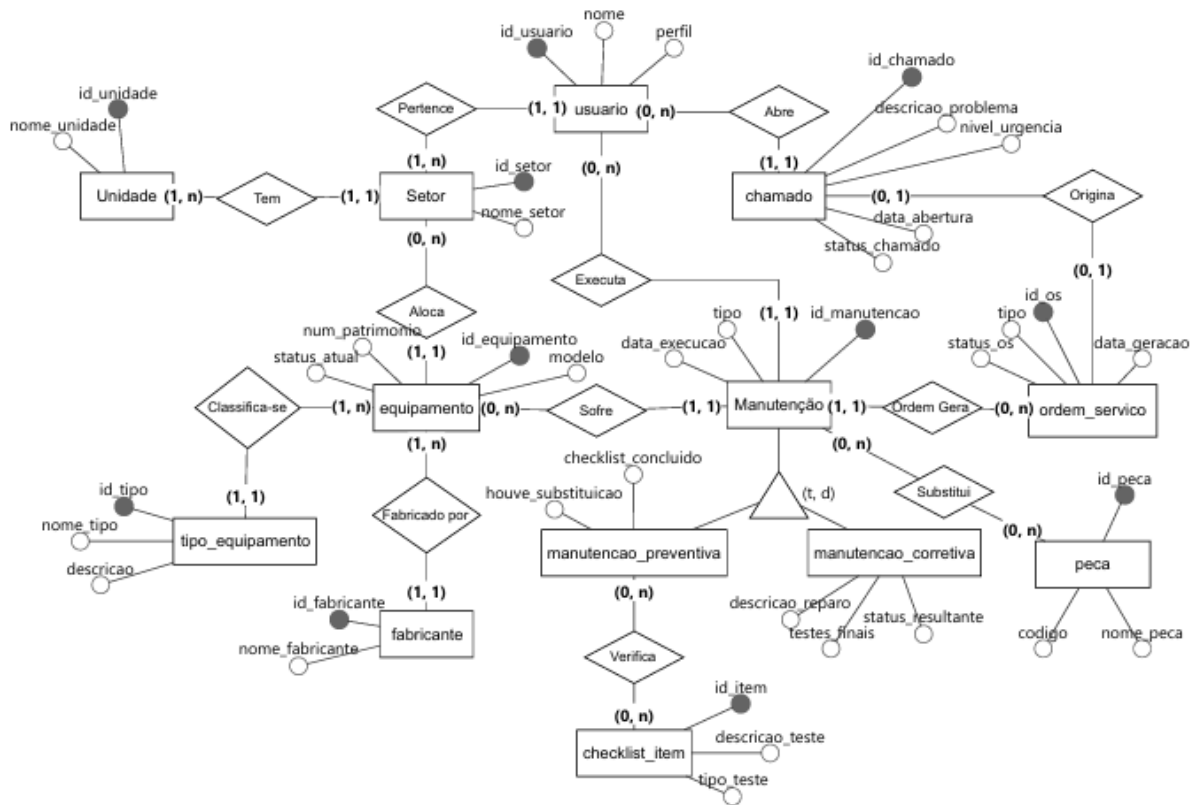


Figura 8: Modelo Conceitual

Modelo Lógico

Após a definição do nível conceitual, apresenta-se o Modelo Lógico do banco de dados. Nesta etapa, as entidades são convertidas em tabelas, definindo-se as chaves primárias (PK), as chaves estrangeiras (FK) e a normalização dos dados até a 3FN. Os relacionamentos 1:N foram materializados como chaves estrangeiras nas tabelas do lado 'muitos' (por exemplo, id_setor em equipamento). Os relacionamentos N:M (peças e checklist) foram resolvidos por meio das tabelas associativas peca_substituida e manutencao_checklist. A especialização da manutenção foi implementada por meio de uma tabela supertipo (manutencao) e duas tabelas subtipo que herdam a chave primária.

Modelo Físico

Nesta etapa, apresenta-se o Modelo Físico do banco de dados. Este modelo detalha a implementação final das tabelas no SGBD SQLite, especificando tipos de dados nativos,

restrições de integridade (constraints) e as views responsáveis pelos dados derivados. O código a seguir reflete a estrutura exata que será instanciada para suportar o sistema HEMC.

Comandos para criação das tabelas

Habilitação de chaves estrangeiras

```
PRAGMA foreign_keys = ON;
```

Construção da tabela unidade

```
CREATE TABLE unidade (  
    id_unidade      INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome_unidade   TEXT NOT NULL  
);
```

Construção da tabela setor

```
CREATE TABLE setor (  
    id_setor      INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome_setor   TEXT NOT NULL,  
    id_unidade   INTEGER NOT NULL,  
    FOREIGN KEY (id_unidade) REFERENCES unidade (id_unidade)  
);
```

Construção da tabela fabricante

```
CREATE TABLE fabricante (  
    id_fabricante  INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome_fabricante TEXT NOT NULL  
);
```

Construção da tabela tipo_equipamento

```
CREATE TABLE tipo_equipamento (  
    id_tipo  INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome_tipo TEXT NOT NULL,  
    descricao TEXT  
);
```

Construção da tabela usuario

```
CREATE TABLE usuario (  
    id_usuario  INTEGER PRIMARY KEY AUTOINCREMENT,  
    nome       TEXT NOT NULL,  
    perfil     TEXT NOT NULL CHECK (perfil IN ('admin', 'enfermeira', 'tecnico')),  
    id_setor   INTEGER NOT NULL,  
    FOREIGN KEY (id_setor) REFERENCES setor (id_setor)  
);
```

Construção da tabela equipamento

```
CREATE TABLE equipamento (  

```

```

id_equipamento    INTEGER PRIMARY KEY AUTOINCREMENT,
num_patrimonio     TEXT NOT NULL UNIQUE,
modelo            TEXT,
num_serie          TEXT,
data_aquisicao      TEXT,
valor_aquisicao     REAL,
grau_criticidade   TEXT NOT NULL CHECK (grau_criticidade IN ('A', 'B', 'C')),
frequencia_preventiva TEXT NOT NULL CHECK (frequencia_preventiva IN ('mensal', 'trimestral')),
status_atual       TEXT NOT NULL DEFAULT 'em_funcionamento'
                    CHECK (status_atual IN ('em_funcionamento', 'em_manutencao',
                    'aguardando_reparo', 'fora_de_operacao')),
id_setor           INTEGER NOT NULL,
id_tipo            INTEGER NOT NULL,
id_fabricante      INTEGER NOT NULL,
FOREIGN KEY (id_setor)      REFERENCES setor (id_setor),
FOREIGN KEY (id_tipo)      REFERENCES tipo_equipamento (id_tipo),
FOREIGN KEY (id_fabricante) REFERENCES fabricante (id_fabricante)
);

```

Construção da tabela chamado

```

CREATE TABLE chamado (
    id_chamado        INTEGER PRIMARY KEY AUTOINCREMENT,
    descricao_problema TEXT NOT NULL,
    nivel_urgencia     TEXT NOT NULL CHECK (nivel_urgencia IN ('A', 'B', 'C')),
    data_abertura      TEXT NOT NULL DEFAULT (datetime('now')),
    status_chamado     TEXT NOT NULL DEFAULT 'aberto'
                    CHECK (status_chamado IN ('aberto', 'em_andamento', 'fechado')),
    id_equipamento    INTEGER NOT NULL,
    id_usuario_abertura INTEGER NOT NULL,
    id_setor           INTEGER NOT NULL,
    FOREIGN KEY (id_equipamento) REFERENCES equipamento (id_equipamento),
    FOREIGN KEY (id_usuario_abertura) REFERENCES usuario (id_usuario),
    FOREIGN KEY (id_setor)      REFERENCES setor (id_setor)
);

```

Construção da tabela ordem_servico

```

CREATE TABLE ordem_servico (
    id_os            INTEGER PRIMARY KEY AUTOINCREMENT,
    tipo             TEXT NOT NULL CHECK (tipo IN ('preventiva', 'corretiva')),
    data_geracao     TEXT NOT NULL DEFAULT (datetime('now')),
    data_programada  TEXT,
    status_os        TEXT NOT NULL DEFAULT 'aberta'
                    CHECK (status_os IN ('aberta', 'em_execucao', 'concluida', 'cancelada')),
    id_equipamento  INTEGER NOT NULL,
    id_chamado       INTEGER,
    FOREIGN KEY (id_equipamento) REFERENCES equipamento (id_equipamento),
    FOREIGN KEY (id_chamado)      REFERENCES chamado (id_chamado)
);

```

Construção da tabela manutencao

```

CREATE TABLE manutencao (
    id_manutencao    INTEGER PRIMARY KEY AUTOINCREMENT,
    tipo             TEXT NOT NULL CHECK (tipo IN ('preventiva', 'corretiva')),
    data_execucao    TEXT NOT NULL DEFAULT (datetime('now')),
    id_equipamento  INTEGER NOT NULL,
    id_responsavel   INTEGER NOT NULL,

```

```

        id_os      INTEGER NOT NULL UNIQUE,
        FOREIGN KEY (id_equipamento) REFERENCES equipamento (id_equipamento),
        FOREIGN KEY (id_responsavel) REFERENCES usuario (id_usuario),
        FOREIGN KEY (id_os)      REFERENCES ordem_servico (id_os)
);

```

Construção da tabela manutencao_preventiva

```

CREATE TABLE manutencao_preventiva (
    id_manutencao      INTEGER PRIMARY KEY,
    houve_substituicao  INTEGER NOT NULL DEFAULT 0 CHECK (houve_substituicao IN (0, 1)),
    checklist_concluido INTEGER NOT NULL DEFAULT 0 CHECK (checklist_concluido IN (0, 1)),
    FOREIGN KEY (id_manutencao) REFERENCES manutencao (id_manutencao) ON DELETE CASCADE
);

```

Construção da tabela manutencao_corretiva

```

CREATE TABLE manutencao_corretiva (
    id_manutencao      INTEGER PRIMARY KEY,
    descricao_reparo   TEXT,
    testes_finais      TEXT,
    status_resultante  TEXT CHECK (status_resultante IN ('em_funcionamento', 'em_manutencao',
        'aguardando_reparo', 'fora_de_operacao')),
    FOREIGN KEY (id_manutencao) REFERENCES manutencao (id_manutencao) ON DELETE CASCADE
);

```

Construção da tabela peca

```

CREATE TABLE peca (
    id_pecas      INTEGER PRIMARY KEY AUTOINCREMENT,
    nome_pecas   TEXT NOT NULL,
    codigo       TEXT
);

```

Construção da tabela peca_substituida

```

CREATE TABLE peca_substituida (
    id_manutencao  INTEGER NOT NULL,
    id_pecas       INTEGER NOT NULL,
    quantidade     INTEGER NOT NULL DEFAULT 1 CHECK (quantidade > 0),
    PRIMARY KEY (id_manutencao, id_pecas),
    FOREIGN KEY (id_manutencao) REFERENCES manutencao (id_manutencao) ON DELETE CASCADE,
    FOREIGN KEY (id_pecas)      REFERENCES peca (id_pecas)
);

```

Construção da tabela checklist_item

```

CREATE TABLE checklist_item (
    id_item      INTEGER PRIMARY KEY AUTOINCREMENT,
    descricao_teste TEXT NOT NULL,
    tipo_teste   TEXT NOT NULL CHECK (tipo_teste IN ('seguranca', 'calibracao'))
);

```

Construção da tabela manutencao_checklist

```
CREATE TABLE manutencao_checklist (
    id_manutencao INTEGER NOT NULL,
    id_item        INTEGER NOT NULL,
    resultado      TEXT NOT NULL CHECK (resultado IN ('aprovado', 'reprovado', 'nao_aplicavel')),
    observacao     TEXT,
    PRIMARY KEY (id_manutencao, id_item),
    FOREIGN KEY (id_manutencao) REFERENCES manutencao_preventiva (id_manutencao) ON DELETE CASCADE,
    FOREIGN KEY (id_item)       REFERENCES checklist_item (id_item)
);
```

Indicadores e alertas (views)

Os indicadores de desempenho (REQ16) e os alertas de vencimento (REQ9 e REQ10) são dados derivados. Por isso, não são armazenados em tabelas: são implementados como views, calculadas a partir das manutenções e das ordens de serviço.

View de indicadores por equipamento

```
CREATE VIEW vw_indicadores_equipamento AS
SELECT
    e.id_equipamento,
    e.num_patrimonio,
    e.grau_criticidade,
    e.status_atual,
    COUNT(m.id_manutencao)           AS total_manutencoes,
    SUM(CASE WHEN m.tipo = 'preventiva' THEN 1 ELSE 0 END) AS total_preventivas,
    SUM(CASE WHEN m.tipo = 'corretiva' THEN 1 ELSE 0 END) AS total_corretivas,
    MAX(m.data_execucao)             AS ultima_manutencao
FROM equipamento e
LEFT JOIN manutencao m ON m.id_equipamento = e.id_equipamento
GROUP BY e.id_equipamento;
```

View de alertas de vencimento

```
CREATE VIEW vw_alertas_vencimento AS
SELECT
    os.id_os,
    os.id_equipamento,
    e.num_patrimonio,
    e.grau_criticidade,
    os.data_programada,
    CASE WHEN os.data_programada < date('now') THEN 'VENCIDO' ELSE 'PROXIMO' END AS situacao
FROM ordem_servico os
JOIN equipamento e ON e.id_equipamento = os.id_equipamento
WHERE os.tipo = 'preventiva'
    AND os.status_os IN ('aberta', 'em_execucao')
    AND os.data_programada <= date('now', '+7 days');
```

Observação sobre as chaves estrangeiras

No SGBD SQLite, as chaves estrangeiras são declaradas diretamente na criação das tabelas (cláusula FOREIGN KEY), como demonstrado acima, não sendo necessário o uso de comandos ALTER TABLE posteriores. É importante destacar que o SQLite só aplica a integridade referencial quando a diretiva PRAGMA foreign_keys = ON é executada a cada

conexão com o banco; portanto, a aplicação em FastAPI deve garantir essa configuração na abertura de cada conexão.

Regras de Segurança

Esta seção descreve a política de Controle de Acesso do sistema HEMC, fundamentada no princípio do menor privilégio. A estrutura de permissões foi desenhada para segregar as funções de cada perfil de usuário, garantindo a integridade do histórico de manutenções e a segurança dos dados.

Controle de acesso e permissões

O Administrador possui controle total sobre o cadastro e a configuração dos equipamentos. A Enfermeira possui acesso restrito à abertura de chamados. O Funcionário Técnico atua na execução das manutenções e no registro das intervenções. A tabela a seguir resume as permissões por grupo.

Grupos de usuários	Permissões	Descrição
Administrador	Cadastrar, editar, excluir e consultar equipamentos; configurar criticidade e frequência; visualizar indicadores; gerenciar usuários.	Acesso total à estrutura de gestão dos equipamentos e à configuração do sistema.
Enfermeira	Abrir chamados de manutenção corretiva; consultar equipamentos.	Acesso restrito à comunicação de falhas; não executa nem registra manutenções.
Funcionário Técnico	Abrir chamados; executar manutenções preventivas e corretivas; registrar peças, checklist e testes; atualizar status do equipamento.	Acesso às funcionalidades de execução e registro das intervenções técnicas.

2.7 Ferramenta de hospedagem do projeto

A plataforma GitHub foi utilizada como ferramenta de hospedagem e compartilhamento do projeto, permitindo o armazenamento do código-fonte, controle de versões e colaboração entre os integrantes da equipe.

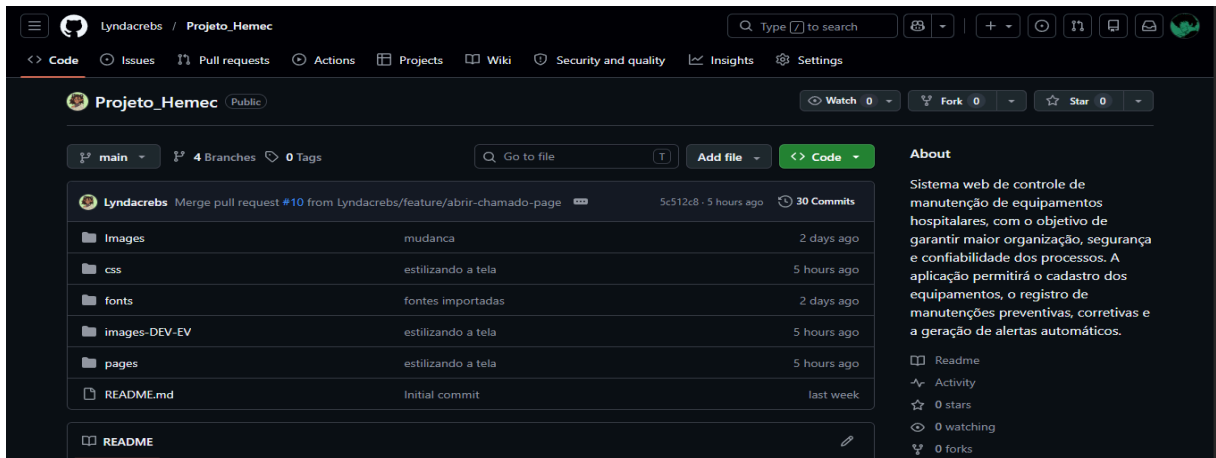


Figura 9: Projeto no Github

Por meio do GitHub, foi possível realizar commits das alterações desenvolvidas no Front-End, acompanhar modificações no sistema e organizar o progresso do projeto de forma centralizada. A plataforma também auxiliou no trabalho colaborativo, possibilitando o uso de branches para desenvolvimento de funcionalidades separadas e integração do código ao projeto principal.

Além disso, o uso do GitHub está relacionado com práticas da Engenharia de Software, como versionamento, rastreabilidade de alterações e integração contínua do desenvolvimento, contribuindo para maior organização e segurança no gerenciamento do sistema.

2.8 Estrutura do Projeto

O projeto HEMC foi desenvolvido utilizando uma estrutura modular, separando as responsabilidades de cada camada da aplicação para facilitar a manutenção, organização e evolução do sistema.

O arquivo **database.py** é responsável pela configuração da conexão com o banco de dados SQLite, criação das sessões utilizadas pela aplicação e definição da classe base utilizada pelos modelos SQLAlchemy. Também realiza a configuração das chaves estrangeiras do banco de dados.

O arquivo **main.py** funciona como o ponto central da aplicação FastAPI. Nele são registradas as rotas da API, configurados os arquivos estáticos do frontend e executadas rotinas de inicialização, como a verificação de usuários e a geração automática de ordens de serviço preventivas.

A pasta **models/** contém os modelos SQLAlchemy que representam as tabelas do banco de dados. Entre as principais entidades estão equipamentos, chamados, ordens de serviço, manutenções, usuários, sessões e tabelas de catálogo.

A pasta **routes/** reúne os roteadores da API, responsáveis por implementar as regras de negócio e disponibilizar os endpoints do sistema. Cada arquivo trata um domínio específico, como autenticação, equipamentos, chamados, ordens de serviço, manutenções e usuários.

A pasta **schemas/** contém os modelos Pydantic utilizados para validação e serialização dos dados enviados e recebidos pela API. Esses schemas garantem maior segurança e consistência na comunicação entre frontend e backend.

A pasta **static/** concentra todos os arquivos do frontend. Nela estão armazenadas as páginas HTML, arquivos CSS, scripts JavaScript e recursos visuais utilizados pela interface do sistema.

Por fim, o arquivo **hemc_schema.sql** documenta a estrutura completa do banco de dados, incluindo tabelas, relacionamentos e restrições.

A arquitetura adotada segue o padrão **MVC adaptado para APIs**, onde os modelos representam os dados, as rotas atuam como controladores, os schemas definem os contratos de comunicação e o frontend funciona como a camada de visualização, comunicando-se com o backend por meio de requisições HTTP.

2.9 Desenvolvimento FrontEnd

HTML

O frontend do HEMC é composto por diversas páginas HTML, cada uma responsável por uma funcionalidade específica do sistema, como autenticação, dashboard, gerenciamento de equipamentos, abertura de chamados, controle de ordens de serviço, registro de manutenções e visualização do perfil do usuário. Todas as páginas estão organizadas na pasta **static/pages/**.

As páginas que exigem autenticação compartilham uma estrutura padrão composta por uma barra lateral de navegação, contendo os principais módulos do sistema, e uma área de conteúdo principal onde são exibidas as informações e funcionalidades de cada tela. Essa padronização contribui para uma navegação mais intuitiva e consistente.

Os formulários desempenham papel fundamental no sistema, sendo utilizados para cadastro de equipamentos, abertura de chamados e registro de manutenções. Os campos são preenchidos dinamicamente por meio de JavaScript, garantindo que apenas informações válidas e previamente cadastradas sejam apresentadas ao usuário.

A navegação entre as páginas é realizada por meio de links HTML convencionais, sem a utilização de frameworks de roteamento. Cada página carrega seus próprios arquivos CSS e JavaScript, mantendo a organização e a separação das responsabilidades dentro do projeto.

CSS

O sistema utiliza uma identidade visual padronizada baseada em tons de azul, buscando transmitir organização e profissionalismo. O azul escuro (#0f1b65) é utilizado na barra lateral, títulos e elementos de destaque, enquanto o azul médio (#2155fc) é aplicado em botões e componentes interativos. O fundo das páginas utiliza um tom de azul claro (#f3f9fc), proporcionando conforto visual, enquanto o branco (ffffff) é empregado nos cards e áreas de conteúdo para melhorar a legibilidade.

A tipografia é composta pelas fontes **Montserrat** e **Inter**, carregadas localmente. A Montserrat é utilizada em títulos, textos e rótulos de formulários, enquanto a Inter é aplicada principalmente em indicadores numéricos do dashboard devido à sua alta legibilidade.

O layout das páginas foi desenvolvido utilizando **CSS Flexbox**, permitindo uma organização flexível dos componentes. A barra lateral permanece fixa durante a navegação, enquanto o conteúdo principal se adapta ao espaço disponível. Além disso, foram utilizadas regras de responsividade com **Media Queries**, garantindo que os elementos sejam reorganizados adequadamente em diferentes tamanhos de tela.

Os indicadores de criticidade e status seguem um padrão visual baseado em cores: vermelho para situações críticas, amarelo para atenção e azul ou verde para estados normais ou operacionais. Essa padronização facilita a identificação rápida das informações pelos usuários.

Por fim, elementos como barras de progresso, formulários e botões utilizam recursos de Flexbox e transições visuais para proporcionar uma interface mais organizada, intuitiva e agradável durante a utilização do sistema.

2.10 Desenvolvimento BackEnd

FastAPI

O FastAPI é um framework web moderno para Python, desenvolvido para criação de APIs de alta performance. Sua principal característica é a utilização de type hints do Python para gerar automaticamente validação de dados, serialização de respostas e documentação interativa da API. O framework é construído sobre o padrão ASGI (Asynchronous Server Gateway Interface), permitindo operações eficientes e maior desempenho no processamento de requisições.

No projeto HEMC, o FastAPI foi escolhido por três motivos principais. O primeiro é sua integração nativa com o Pydantic, permitindo declarar schemas diretamente nas funções de rota, eliminando grande parte da validação manual de dados. O segundo motivo é a documentação automática gerada pelo próprio framework, disponível em [/docs](#) através do Swagger UI e em [/openapi.json](#) através do padrão OpenAPI. Essa funcionalidade facilitou significativamente os testes dos endpoints durante o desenvolvimento do sistema. O terceiro motivo é a clareza do código, já que cada endpoint é implementado como uma função Python comum, utilizando tipagem explícita nos parâmetros e retornos, tornando a estrutura mais organizada e legível.

Rotas e Endpoints

A API do HEMC é organizada em sete roteadores principais, registrados no arquivo **main.py** utilizando o prefixo **/api**. Cada roteador representa um domínio específico do sistema e é criado através da classe **APIRouter**, mecanismo utilizado pelo FastAPI para modularizar aplicações maiores.

O roteador de equipamentos, localizado em **routes/equipamento.py**, utiliza o prefixo **"/equipamentos"** e implementa o CRUD completo da entidade equipamento. O endpoint **GET /api/equipamentos/** retorna todos os equipamentos cadastrados, enquanto **GET /api/equipamentos/{id_equipamento}** retorna um equipamento específico com base no identificador informado. O endpoint **POST /api/equipamentos/** cria novos registros, **PUT /api/equipamentos/{id_equipamento}** atualiza os dados existentes e **DELETE /api/equipamentos/{id_equipamento}** remove um equipamento do banco de dados.

Essa estrutura segue o padrão REST, no qual o método HTTP define a operação e a URL identifica o recurso manipulado.

O arquivo **routes/chamado.py** demonstra a utilização do FastAPI para implementar regras de negócio diretamente dentro das funções de rota. O endpoint **POST /api/chamados/** não apenas cria um chamado no banco de dados, mas também gera automaticamente uma Ordem de Serviço corretiva vinculada ao chamado e altera o status do equipamento para **"aguardando_reparo"** dentro da mesma transação.

O arquivo **routes/manutencao.py** implementa dois endpoints especializados:

```
POST /api/manutencoes/preventiva
POST /api/manutencoes/corretiva
```

A separação ocorre devido às diferenças estruturais entre os dois tipos de manutenção. A preventiva exige preenchimento de checklist técnico e registro de peças substituídas, enquanto a corretiva exige descrição do reparo, testes executados e definição do status final do equipamento.

Cada endpoint recebe dependências automaticamente através do sistema de Dependency Injection do FastAPI. Entre as principais dependências utilizadas estão:

```
db: Session = Depends(get_db)
```

e:

```
_: Usuario = Depends(get_current_user)
```

O **Depends** é o mecanismo utilizado pelo FastAPI para executar funções auxiliares antes da execução do endpoint principal. Dessa forma, o sistema consegue fornecer automaticamente uma sessão de banco de dados e validar a autenticação do usuário antes de processar qualquer operação.

Os métodos HTTP utilizados seguem uma semântica padronizada:

- GET: consulta de dados
- POST: criação de registros
- PUT: atualização completa

- PATCH: atualização parcial
- DELETE: remoção de registros

Os códigos de status HTTP também seguem o padrão REST:

- 200 OK: operação realizada com sucesso
- 201 Created: recurso criado
- 204 No Content: exclusão concluída
- 400 Bad Request: dados inválidos
- 401 Unauthorized: usuário não autenticado
- 403 Forbidden: usuário sem permissão
- 404 Not Found: recurso inexistente

Fluxo do Backend

O fluxo de processamento do backend pode ser compreendido observando o caminho percorrido por uma requisição desde o frontend até o banco de dados.

Quando o frontend envia um **POST /api/chamados/** contendo:

```
{
  "descricao_problema": "Alarme sonoro com falha",
  "nivel_urgencia": "A",
  "id_equipamento": 3,
  "id_usuario_abertura": 1,
  "id_setor": 2
}
```

o FastAPI identifica automaticamente qual endpoint deve processar a requisição.

Antes da execução da função principal, o framework resolve todas as dependências declaradas na assinatura da rota. A função **get_db()** cria uma sessão de banco de dados e a dependência **require_enfermeira** valida o token do usuário e verifica se o perfil possui permissão para abrir chamados.

A função **get_current_user**, localizada em **routes/auth.py**, extrai o token do cabeçalho:

```
Authorization: Bearer <token>
```

Em seguida, o sistema consulta a tabela **sessao** no banco de dados. Caso o token não exista, o backend retorna automaticamente:

```
HTTPException(status_code=401)
```

Se o usuário autenticado não possuir o perfil necessário, o sistema retorna:

```
HTTPException(status_code=403)
```

Após validar a autenticação, o FastAPI utiliza o Pydantic para validar o corpo da requisição através do schema **ChamadoCreate**. O framework verifica tipos de dados, campos obrigatórios e valores permitidos. Caso exista qualquer inconsistência, o FastAPI retorna automaticamente o erro **422 Unprocessable Entity**.

Com os dados validados, a função de rota é executada. O backend cria um objeto **Chamado**, adiciona-o à sessão com:

```
db.add(chamado)
```

Em seguida, utiliza:

```
db.flush()
```

para obter o **id_chamado** antes da confirmação definitiva da transação. Esse identificador é necessário para criar a Ordem de Serviço vinculada ao chamado recém-criado.

Após criar a OS corretiva e atualizar o status do equipamento, o sistema confirma todas as alterações utilizando:

```
db.commit()
```

O método **commit()** é responsável por tornar permanentes as alterações realizadas no banco de dados. Caso qualquer erro ocorra antes desse ponto, toda a transação é revertida automaticamente.

Por fim, o backend executa:

```
db.refresh(chamado)
```

para atualizar o objeto com os dados finais armazenados no banco, incluindo IDs e datas geradas automaticamente.

O FastAPI então serializa o objeto utilizando o schema **ChamadoOut** e retorna a resposta HTTP **201 Created**.

SQLAlchemy

O SQLAlchemy é a biblioteca ORM (Mapeamento Objeto-Relacional) utilizada no HEMC para realizar a comunicação entre o Python e o banco de dados SQLite. Um ORM permite manipular tabelas através de objetos Python, reduzindo a necessidade de escrever comandos SQL manualmente.

Todos os modelos do sistema herdam de **database.Base**, uma instância de **DeclarativeBase**. Essa herança permite que o SQLAlchemy associe cada classe Python a uma tabela do banco de dados.

O modelo **Equipamento**, localizado em **models/equipamento.py**, exemplifica essa estrutura:

```
class Equipamento(Base):
    __tablename__ = "equipamento"

    id_equipamento = Column(Integer, primary_key=True, autoincrement=True)
    num_patrimonio = Column(String, nullable=False, unique=True)
    grau_criticidade = Column(String, nullable=False)
    status_atual = Column(String, nullable=False, default="em_funcionamento")

    id_setor = Column(Integer, ForeignKey("setor.id_setor"), nullable=False)
    id_tipo = Column(Integer, ForeignKey("tipo_equipamento.id_tipo"), nullable=False)
    id_fabricante = Column(Integer, ForeignKey("fabricante.id_fabricante"),
        nullable=False)

    __table_args__ = (
        CheckConstraint(
            "grau_criticidade IN ('A','B','C')",
            name="ck_criticidade"
        ),
    )
```

O atributo **__tablename__** define o nome da tabela no banco. As **ForeignKey** estabelecem os relacionamentos entre tabelas e garantem integridade referencial. As **CheckConstraint** limitam os valores permitidos para determinados campos.

A função **get_db()** em **database.py** gerencia o ciclo de vida da sessão de banco de dados:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

As consultas são realizadas utilizando métodos ORM como:

```
db.query(Classe).filter(condicao).all()
```

e:

```
db.get(Classe, id)
```

A inserção de registros utiliza:

```
db.add(objeto)
```

e a persistência definitiva ocorre com:

```
db.commit()
```

O modelo **Manutencao** utiliza herança de tabelas (Table Inheritance). A tabela principal **manutencao** armazena os dados comuns, enquanto **manutencao_preventiva** e **manutencao_corretiva** armazenam atributos específicos de cada tipo de manutenção.

Schemas Pydantic

O Pydantic é utilizado no HEMC para validar os dados recebidos e enviados pela API. Os schemas estão organizados na pasta **schemas/** e seguem um padrão dividido em três tipos principais:

- Base
- Create
- Out

O schema **Base** define os campos compartilhados. O schema **Create** representa os dados necessários para criação de registros, enquanto o schema **Out** define o formato utilizado nas respostas da API.

O arquivo **schemas/equipamento.py** exemplifica essa estrutura. O schema **EquipamentoBase** define os campos comuns utilizando tipagem explícita:

```
Criticidade = Literal["A", "B", "C"]
Frequencia = Literal["mensal", "trimestral"]
```

Esses **Literal** limitam os valores aceitos, funcionando como validações automáticas.

O schema **EquipamentoCreate** herda diretamente de **EquipamentoBase**, reutilizando todos os campos. Já **EquipamentoUpdate** utiliza campos **Optional**, permitindo atualizações parciais.

O schema **EquipamentoOut** adiciona o identificador do equipamento e configura:

```
model_config = {"from_attributes": True}
```

Essa configuração permite ao Pydantic converter diretamente objetos SQLAlchemy em respostas JSON.

O arquivo **schemas/manutencao.py** demonstra a validação de estruturas aninhadas:

```
class ManutencaoPreventivaCreate(BaseModel):
    id_os: int
    id_equipamento: int
    id_responsavel: int
    houve_substituicao: bool = False

    pecas: list[PecaSubstituidaIn] = []
    checklist: list[ChecklistItemIn] = []
```

Quando o frontend envia listas de peças substituídas e itens de checklist, o Pydantic valida automaticamente cada item individualmente antes que os dados sejam processados pelo backend.

2.11 API e Comunicação Entre Camadas

A Application Programming Interface (API) do HEMC é o contrato responsável por definir a comunicação entre o frontend e o backend do sistema. Em termos técnicos, uma API consiste em um conjunto de regras e definições que especifica quais operações podem ser realizadas, quais dados devem ser enviados pelo cliente e quais informações serão retornadas pelo servidor.

O sistema HEMC utiliza o padrão arquitetural REST (Representational State Transfer), baseado no protocolo HTTP e na organização de recursos através de URLs. Nesse modelo, cada recurso do sistema (como equipamentos, chamados, ordens de serviço e manutenções) possui um endereço específico na API. As operações realizadas sobre esses recursos são definidas pelos métodos HTTP, sendo GET utilizado para consultas, POST para criação de registros, PUT para atualização completa, PATCH para atualização parcial e DELETE para remoção de dados.

No HEMC, o estado da aplicação permanece armazenado no servidor, principalmente no banco de dados SQLite. O frontend funciona como cliente da API, enviando requisições sempre que necessita consultar ou modificar informações. Com exceção do token de autenticação armazenado no localStorage do navegador, cada requisição contém todos os dados necessários para ser processada independentemente.

O protocolo HTTP (HyperText Transfer Protocol) é o responsável pela troca de informações entre cliente e servidor. Toda comunicação entre o frontend e o backend ocorre através de mensagens HTTP. O navegador envia uma requisição contendo o método HTTP, a URL desejada, cabeçalhos e, quando necessário, um corpo com dados. O servidor processa a solicitação e retorna uma resposta contendo um código de status, cabeçalhos e os dados solicitados.

Os dados trafegados entre as camadas são serializados no formato JSON (JavaScript Object Notation). O JSON é amplamente utilizado em APIs REST por ser leve, legível e possuir suporte nativo em JavaScript e Python. Um exemplo real utilizado no sistema ocorre durante a abertura de chamados, onde o frontend envia um objeto semelhante ao seguinte:

```
{
  "descricao_problema": "Sensor de SpO2 com leitura incorreta",
  "nivel_urgencia": "B",
  "id_equipamento": 5,
  "id_usuario_abertura": 2,
  "id_setor": 1
}
```

No frontend, a comunicação com a API é centralizada no arquivo **static/js/api.js**, através do objeto **API**. Quando a página de equipamentos é carregada, o JavaScript executa:

```
API.get("/equipamentos/")
```

Internamente, esse método utiliza a função **fetch** do navegador:

```
fetch("/api/equipamentos/", {
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer <token>"
  }
})
```

Nesse momento, o navegador envia uma requisição HTTP GET para o backend FastAPI. O servidor recebe a solicitação, valida o token através da função **get_current_user**, consulta os registros da tabela **equipamento** utilizando SQLAlchemy e retorna uma lista em JSON contendo todos os equipamentos cadastrados.

Após receber a resposta, o frontend converte os dados utilizando **res.json()**, armazena o resultado em memória e atualiza dinamicamente a interface da página. No arquivo **equipamentos.js**, a função **renderizarTabela()** constrói as linhas HTML da tabela utilizando os dados retornados pela API e os insere dentro do elemento **<tbody>**.

Outro exemplo importante é o endpoint **GET /api/indicadores**, utilizado pelo dashboard do sistema. Esse endpoint realiza múltiplas consultas ao banco de dados para calcular indicadores como quantidade total de equipamentos, ordens de serviço abertas, equipamentos em manutenção e taxa de conformidade. Todos esses valores são agrupados em um único objeto JSON e enviados ao frontend em uma única resposta, reduzindo a quantidade de requisições necessárias.

No arquivo **dashboard.js**, os dados são carregados utilizando requisições paralelas:

```
const [ind, ordens] = await Promise.all([
```

```
API.get("/indicadores"),
API.get("/ordens-servico/")
]);
```

Após receber os dados, o JavaScript atualiza os cards, gráficos, barras de progresso e tabelas do dashboard sem necessidade de recarregar a página.

O sistema de autenticação também faz parte da comunicação entre frontend e backend. Quando o usuário realiza login, o frontend envia uma requisição **POST /api/login** contendo nome e senha:

```
{
  "nome": "Carlos Santos",
  "senha": "senha123"
}
```

No backend, a senha é convertida em hash utilizando SHA-256:

```
hashlib.sha256(senha.encode()).hexdigest()
```

O servidor compara esse hash com o valor armazenado no banco de dados. Caso os dados estejam corretos, um token aleatório é gerado utilizando:

```
secrets.token_urlsafe(32)
```

Esse token é armazenado na tabela **sessao** e retornado ao frontend junto com os dados do usuário autenticado.

Após o login, o frontend salva o token no **localStorage** e o envia automaticamente em todas as requisições subsequentes através do cabeçalho Authorization. Dessa forma, o backend consegue identificar qual usuário está realizando cada operação no sistema.

O endpoint **POST /api/logout** realiza o encerramento da sessão. Quando chamado, o backend remove o token da tabela **sessao**, invalidando o acesso do usuário e obrigando um novo login para continuar utilizando o sistema.

Toda essa estrutura demonstra como frontend, backend e banco de dados trabalham de forma integrada no HEMC. O frontend é responsável pela interface e experiência do usuário, o backend implementa as regras de negócio e a API REST funciona como a ponte de comunicação entre essas duas camadas, garantindo organização, modularidade e segurança na troca de informações.

3 RESULTADOS E DISCUSSÕES

A etapa de testes do sistema HEMC teve como principal objetivo validar o funcionamento correto das funcionalidades implementadas, garantindo a integridade dos dados, a comunicação entre as camadas da aplicação e a usabilidade da interface para os usuários finais. Os testes foram realizados manualmente durante o desenvolvimento do sistema, envolvendo principalmente a análise do frontend, do backend e da integração entre ambos.

Os primeiros testes concentraram-se na interface gráfica do sistema. Foram verificadas questões relacionadas ao layout das páginas, alinhamento dos componentes e comportamento responsivo em diferentes resoluções de tela. O frontend foi testado em múltiplos tamanhos de monitor e janelas do navegador para garantir que tabelas, formulários, cards e menus laterais permanecessem organizados e proporcionais, sem quebra de layout ou sobreposição de elementos. Essa verificação foi importante para assegurar uma experiência de uso consistente em diferentes ambientes.

Também foram realizados testes de usabilidade e navegação entre as páginas do sistema. Foram analisados elementos como botões, menus, formulários e mensagens exibidas ao usuário, verificando se todas as ações executavam corretamente suas respectivas funções. Os formulários de cadastro, abertura de chamados e registro de manutenção passaram por testes de preenchimento completo e parcial, garantindo que os campos obrigatórios fossem corretamente validados antes do envio.

Na comunicação entre frontend e backend, foram executados testes para confirmar se a API estava transportando corretamente os dados entre as duas camadas da aplicação. Cada operação realizada pelo usuário na interface (como cadastro de equipamentos, abertura de chamados e registro de manutenções) foi acompanhada pela verificação das requisições HTTP enviadas à API REST. Os testes confirmaram que os endpoints estavam recebendo os dados corretamente, processando as regras de negócio e retornando respostas adequadas ao frontend.

Além da comunicação da API, foram realizados testes de persistência no banco de dados. Após cada operação executada no sistema, verificou-se diretamente no SQLite se os registros estavam sendo armazenados corretamente nas tabelas correspondentes. Foram testadas inserções, atualizações e relacionamentos entre entidades, como a criação automática de Ordens de Serviço após a abertura de chamados e o registro de peças substituídas durante manutenções corretivas.

A parte de autenticação também recebeu atenção especial durante os testes. Foi verificado se as senhas dos usuários estavam sendo armazenadas de forma criptografada utilizando hash SHA-256, garantindo maior segurança dos dados sensíveis. Além disso, foram realizados testes com credenciais inválidas no sistema de login para validar se o backend bloqueava corretamente acessos não autorizados. Os resultados mostraram que o sistema respondia adequadamente com mensagens de erro e impedia o acesso quando usuário ou senha estavam incorretos.

Os resultados obtidos demonstraram que o sistema apresentou funcionamento estável durante os cenários avaliados. A integração entre frontend, backend e banco de dados

ocorreu de forma consistente, sem perda de informações durante as operações testadas. A interface manteve boa organização visual em diferentes resoluções e os mecanismos de autenticação e validação responderam corretamente às situações de erro simuladas. Dessa forma, os testes realizados contribuíram para aumentar a confiabilidade do HEMC e validar sua capacidade de atender às funcionalidades propostas no projeto.

4 CONCLUSÃO

O desenvolvimento do HEMC permitiu aplicar, de forma prática, conceitos fundamentais da engenharia de software, banco de dados, desenvolvimento web e integração entre sistemas. A construção da aplicação possibilitou compreender como frontend, backend e banco de dados trabalham de maneira integrada para atender às necessidades de um ambiente hospitalar, além de proporcionar experiência no uso de tecnologias como FastAPI, SQLAlchemy, Pydantic, SQLite e JavaScript.

Durante o desenvolvimento, a equipe enfrentou diversas dificuldades técnicas relacionadas a conceitos que ainda estavam em processo de aprendizado, como funcionamento de APIs REST, endpoints, rotas, comunicação entre frontend e backend, modelagem em SQLite e utilização do ORM SQLAlchemy com FastAPI. A compreensão da estrutura de requisições HTTP, autenticação, validação de dados e organização das rotas também representou um desafio importante ao longo do projeto. Entretanto, essas dificuldades foram gradualmente superadas através de estudos, testes práticos e experimentação durante a construção do sistema.

Os testes realizados demonstraram que o sistema apresentou funcionamento satisfatório na maior parte das funcionalidades implementadas, especialmente na comunicação entre as camadas da aplicação, no armazenamento das informações e no fluxo de manutenção de equipamentos. A interface também apresentou boa estabilidade visual em diferentes resoluções e os mecanismos de autenticação funcionaram corretamente durante os cenários testados.

Apesar disso, alguns pontos de melhoria foram identificados durante os testes. Em determinados casos, algumas funcionalidades e elementos da interface que deveriam estar visíveis apenas para técnicos ainda apareciam para outros perfis de usuário, demonstrando a necessidade de um controle de permissões mais refinado também no frontend, além do backend. Também foram observados pequenos ajustes necessários relacionados à organização visual e validação de determinadas ações do sistema.

Ao longo do desenvolvimento, ferramentas de inteligência artificial também foram utilizadas como apoio complementar para pesquisa, esclarecimento de dúvidas e auxílio na resolução de problemas específicos de programação. Esse suporte contribuiu para acelerar parte do aprendizado técnico e auxiliar na compreensão de conceitos mais complexos, sem substituir o processo de estudo, implementação e adaptação realizado pela equipe.

Dessa forma, o projeto não apenas resultou em um sistema funcional, mas também proporcionou um importante aprendizado técnico e prático para os integrantes envolvidos.

Como trabalhos futuros, pretende-se aprimorar o sistema de permissões, fortalecer ainda mais a segurança da aplicação, melhorar a responsividade da interface e adicionar novas funcionalidades, tornando o HEMC uma solução ainda mais robusta e adequada para ambientes hospitalares reais.

REFERÊNCIAS

Lista de referências em ordem alfabética

- Api utilizada para backend. FastAPI, 2026. Disponível em: <<https://fastapi.tiangolo.com/>> Acesso em: 15 maio 2026.
- Protótipo do sistema desenvolvido no Figma. Figma, 2026. Disponível em: <<https://www.figma.com/design/hPdfn8CLlIc24zY7LlFXns/HEMEC---Web-Site?node-id=48-6&t=i5AlqXzFewAymwP3-1>>. Acesso em: 15 maio 2026.